

Este módulo usa o SDK do .NET 8.0. Verifique se você tem o .NET 8.0 instalado executando o seguinte comando em seu terminal de comando preferencial:

CLI do .NET

```
dotnet --list-sdks
```

Uma saída semelhante ao seguinte exemplo aparece:

Console

```
6.0.317 [C:\Program Files\dotnet\sdk]  
7.0.401 [C:\Program Files\dotnet\sdk]  
8.0.100 [C:\Program Files\dotnet\sdk]
```

Verifique se uma versão que começa com `8` está listada. Se nenhum estiver listado ou o comando não for encontrado, [instale o SDK do .NET 8.0 mais recente](#).

Criar e explorar um projeto de API Web

Para configurar o projeto do .NET a fim de trabalhar com a API Web, usaremos o Visual Studio Code. O Visual Studio Code inclui um terminal integrado, o que facilita a criação de projetos. Se você não quiser usar um editor de código, poderá executar os comandos deste módulo em um terminal.

1. No Visual Studio Code, escolha **Arquivo>Abrir Pasta**.
2. Crie uma pasta chamada **ContosoPizza** no local de sua escolha e clique em **Selecionar Pasta**.
3. No Visual Studio Code, abra o terminal integrado selecionando **Exibir>Terminal** no menu principal.
4. Na janela do terminal, copie e cole o seguinte comando:

CLI do .NET

```
dotnet new webapi -controllers -f net8.0
```

Esse comando cria os arquivos de um projeto básico de API Web que usa controladores, juntamente com um arquivo de projeto em C#

chamado *ContosoPizza.csproj* que retornará uma lista de previsões meteorológicas. Caso obtenha um erro, verifique se o [SDK do .NET 8](#) está instalado.

Importante

Os projetos de API Web são protegidos com `https` por padrão. Se você tiver problemas, [configure o certificado de desenvolvimento HTTPS do ASP.NET Core](#).

Você poderá receber um prompt do Visual Studio Code para adicionar ativos para depurar o projeto. Selecione **Sim** na caixa de diálogo.

O comando usa um modelo de projeto ASP.NET Core, com o alias *webapi*, para gerar o scaffolding de um projeto de API Web baseado em C#. Um diretório *ContosoPizza* é criado. Esse diretório contém um projeto ASP.NET Core em execução no .NET. O nome do projeto corresponde ao nome do diretório *ContosoPizza*.

Agora você deve ter acesso a esses arquivos e diretórios:

Bash

```
- | Controllers
- | obj
- | Properties
- | appsettings.Development.json
- | appsettings.json
- | ContosoPizza.csproj
- | ContosoPizza.http
- | Program.cs
- | WeatherForecast.cs
```

5. Examine os seguintes arquivos e diretórios:

Nome	Descrição
<i>Controllers/</i>	Contém classes com

	métodos públicos expostos como pontos de extremidade de HTTP.
<i>Program.cs</i>	Configura serviços e o pipeline de solicitação HTTP do aplicativo e contém o ponto de entrada gerenciado do aplicativo.
<i>ContosoPizzeria.csproj</i>	Contém metadados de configuração para o projeto.
<i>ContosoPizzeria.http</i>	Isso contém a configuração para testar as

	APIs REST diretamen te do Visual Studio Code.
--	---

Compilar e testar a API Web

1. Execute o seguinte comando da CLI do .NET Core no shell de comando:

CLI do .NET

```
dotnet run
```

O comando anterior:

Quando o projeto for criado, uma porta de 5000 a 5300 será selecionada para HTTP e de 7000 a 7300 para HTTPS. Você pode facilmente alterar as portas que você usou durante o desenvolvimento com a edição do arquivo *launchSettings.json* do projeto. Esse módulo usa a URL segura `localhost` que começa com `https`.

Você deve obter uma saída semelhante à seguinte, indicando que seu aplicativo está em execução:

Console

```
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7294
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5118
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
```

Se você estiver executando esse aplicativo no computador, será possível direcionar um navegador para o link HTTPS exibido na saída (no caso anterior, `https://localhost:7294`) para exibir a página resultante. Lembre-se dessa porta, pois você a usará em todo o módulo em que `{PORT}` for usado.

Importante

Verifique a saída do terminal se você encontrar algum comportamento inesperado. Se o build falhar ou se ocorrerem outros erros, as informações do arquivo de log ajudarão a solucionar os problemas. Ao fazer alterações no código, você precisará interromper a API Web selecionando Ctrl+C no teclado e executando o comando `dotnet run` novamente.

- Localiza o arquivo de projeto no diretório atual.
- Recupera e instala as dependências de projeto necessárias para este projeto.
- Compila o código do projeto.
- Hospeda a API Web com o servidor Web Kestrel do ASP.NET Core em um ponto de extremidade HTTP e HTTPS.

2. Abra um navegador da Web e acesse:

Bash

```
https://localhost:{PORT}/weatherforecast
```

Você deverá ver uma saída semelhante a esse exemplo:

JSON

```
[
  {
    "date": "2021-11-09T20:36:01.4678814+00:00",
    "temperatureC": 33,
    "temperatureF": 91,
    "summary": "Scorching"
  },
  {
    "date": "2021-11-09T20:36:01.4682337+00:00",
    "temperatureC": -8,
```

```
        "temperatureF": 18,  
        "summary": "Cool"  
    },  
    // ...  
]
```

Opcional: Explorar com arquivos .http

O **ContosoPizza.http** está incluído no projeto e esse arquivo é usado para testar pontos de extremidade de API por meio de um formato padrão. Há suporte para arquivos **.http** em vários IDEs (ambientes de desenvolvimento integrado), incluindo o Visual Studio e dentro do Visual Studio Code com a extensão REST de cliente instalada.

1. Abra o arquivo ContosoPizza.http.

Em alguns IDEs esse arquivo é pré-configurado com as variáveis de **@ContosoPizza_HostAddress** e um comando **GET** chamando **/weatherforecast/** que aceita **application/json**.

2. Se ele estiver presente em seu arquivo, selecione o comando **Sent Request** acima de **GET**, que envia uma solicitação para o serviço em execução.

Chamar esse comando exibe uma janela de resposta com saída semelhante à vista no navegador:

Saída

```
HTTP/1.1 200 OK  
Connection: close  
Content-Type: application/json; charset=utf-8  
Date: Wed, 17 Jan 2024 16:46:40 GMT  
Server: Kestrel  
Transfer-Encoding: chunked  
  
[  
  {  
    "date": "2024-01-18",  
    "temperatureC": -2,  
    "temperatureF": 29,  
    "summary": "Warm"
```

```
},  
{  
  "date": "2024-01-19",
```

Opcional: Explorar APIs com o REPL HTTP da linha de comando

1. Abra um novo terminal integrado do Visual Studio Code selecionando **Terminal>Novo Terminal** no menu principal e execute o seguinte comando:

CLI do .NET

```
dotnet tool install -g Microsoft.dotnet-httprepl
```

O comando anterior instala a ferramenta de linha de comando .NET HTTP Read-Eval-Print Loop (REPL) que você usa para fazer solicitações HTTP para a API da Web.

2. Conecte-se à API Web executando o seguinte comando:

CLI do .NET

```
httprepl https://localhost:{PORT}
```

Outra opção é executar o seguinte comando a qualquer momento enquanto `HttpRepl` estiver em execução:

CLI do .NET

```
connect https://localhost:{PORT}
```

Dica

Se a `HttpRepl` ferramenta avisar **Não é possível encontrar uma descrição OpenAPI**, a causa mais provável é um certificado de desenvolvimento não confiável. `HttpRepl` requer uma conexão confiável. Antes de continuar, você **precisa configurar seu sistema para confiar no certificado de desenvolvimento** com `dotnet dev-certs https --trust`

3. Explore os pontos de extremidade disponíveis executando o seguinte comando:

CLI do .NET

```
ls
```

O comando anterior detecta todas as APIs disponíveis no ponto de extremidade conectado e as lista, como na seguinte saída:

Saída

```
https://localhost:{PORT}/> ls
.
WeatherForecast [GET]
```

4. Vá até o ponto de extremidade `WeatherForecast` executando o seguinte comando:

CLI do .NET

```
cd WeatherForecast
```

O comando anterior mostra uma saída das APIs disponíveis para o ponto de extremidade `WeatherForecast`:

Saída

```
https://localhost:{PORT}/> cd WeatherForecast
/WeatherForecast [GET]
```

5. Faça uma solicitação `GET` no `HttpRepl` usando o seguinte comando:

CLI do .NET

```
get
```

O comando anterior faz uma solicitação `GET` semelhante a ir para o ponto de extremidade no navegador:

Saída

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Fri, 02 Apr 2021 17:31:43 GMT
Server: Kestrel
Transfer-Encoding: chunked
[
  {
    "date": 4/3/2021 10:31:44 AM,
    "temperatureC": 13,
    "temperatureF": 55,
    "summary": "Sweltering"
  },
  {
    "date": 4/4/2021 10:31:44 AM,
    "temperatureC": -13,
    "temperatureF": 9,
    "summary": "Warm"
  }
]
```

6. Saia da sessão `HttpRepl` atual usando o seguinte comando:

CLI do .NET

```
exit
```

7. Retorne ao terminal `dotnet` na lista suspensa no Visual Studio Code.

Desligue a API Web selecionando Ctrl+C no teclado.

Agora que você criou a API Web, é possível modifica-la para atender às necessidades da API Web de pizza.